JACOB SPREADBOROUGH

UNIVERSITY OF YORK

js3952@york.ac.uk

Cross Platform Mobile Audio App Development - Final Report

3rd Year Electronic Engineering with Music Technology Systems,

Department of Physics, Engineering, and Technology, University of York

Supervisors:

Andy Hunt, Frank Stevens

Contents

1 Introduction **2**

2 Literature Review **3**
2.1 Historical Context and Current Trends 3
2.2 Cross-Platform Approach Comparison 5
2.3 Audio Applications 6

3 Aims and Objectives **9**
3.1 Aims 9
3.2 Objectives 9

4 Design **10**
4.1 Evaluation 10
4.1.1 Framework Criteria 10
4.1.2 Audio Criteria 11

5 Implementation **12**

6 Testing **13**

7 Evaluation and Further Work **14**
7.1 Evaluation 14
7.2 Further Work 14

8 Recommendation **15**

A References **16**

B Figures **18**

1 Introduction

Since their inception, mobile phones have become more and more ubiquitous, becoming a tool that is required for everyday life [1].

Mobile devices are involved in almost every aspect of every day life, and have evolved to handle a huge range of tasks. Among these, audio is one of the most important, as it is required for the communication and entertainment capabilities of mobile devices.

Mobile audio development is constrained mainly by two factors: the speed and quality required for audio processing, and the size and power constraints of a small device with limited battery life. These contstraints are made ever more challenging when attempting to build a single application for multiple mobile phone types, as the methods of controlling audio often involve low level concepts to meet the requirements of the aforementioned factors.

When discussing cross-platform development, the platforms being developed for must be established. In mobile devices, the primary options are Android and iOS [2], with others such as Samsung and Windows falling behind [3]. In this project, the focus will be on development for Android and iOS devices, as together they account for over 90% of mobile devices in use. If companies want their apps to be able to reach as many users as possible, then they must develop them for both Android and iOS devices. To meet the need to create apps for multiple mobile operating systems, while keeping the cost of development low, then an approach is needed that allows for the development of a single codebase that will create similar apps for the different operating systems.

Despite the ubiquity of mobile devices, and the specific challenges posed by audio development, there is little information in the literature on how a developer should approach this specific problem. This project serves to provide an outline a comprehensive evaluation and comparision of the tools available for cross-platform audio development, and to provide recommendations for developers based on their specific situation.

The rest of this paper is structured as follows: Section 2 covers some significant previous work done regarding this topic. Section 3 outlines the goals of this project, and how they will be achieved Section 4 provides an overview of the sample app's design, along with the evaluation methodology used to compare development frameworks. Section 5 is a technical report detailing the sample apps that were implemented using the cross-platform frameworks. Section 6 details the processes and results of user testing, providing insight into the design and implementation process, as well as the final results of using each framework. Section 7 summarises this project's results and findings, and gives a call to action for those in the field of mobile audio development. Section 8 provides recommendations for a engineers looking to develop mobile audio applications depending on their situation and development requirements.

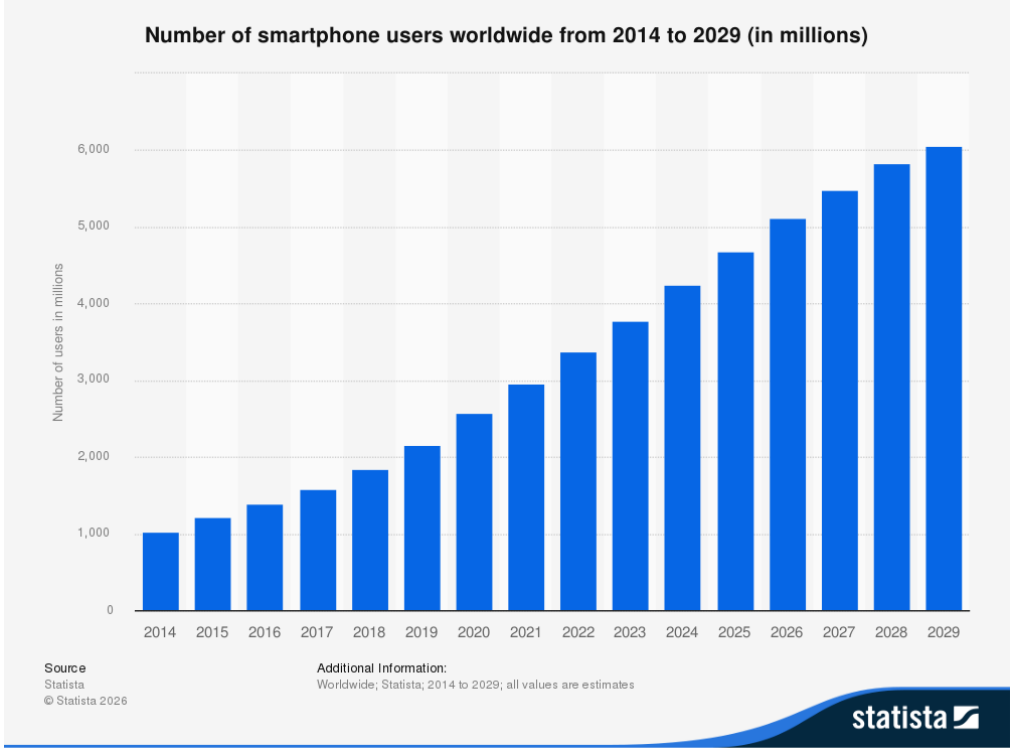

Figure 1: Global cellphone usage from 2019-2029. Data and graphic from [1].

2 Literature Review

'Cross-platform mobile development' refers to the practice of creating applications that will run on a mobile phone regardless of its operating system. This term is often used in contrast to 'native' apps, which are built for only one operating system, using the development resources provided by that system. Native apps have the benefit of having a unified look and feel that is created by platform-specific UI and design principles [4, 5]. The challenge of cross-platform development is creating a method to write one codebase that can be used to create native-passing apps for multiple platforms, instead of having to write separate native apps using different languages and frameworks.

This problem has been around for as long as mobile phone operating systems have existed, and has become more and more relevant as the demand for mobile applications has grown, and along with it the demand for applications that look and feel like native apps.

2.1 Historical Context and Current Trends

Throughout the history of cross-platform mobile development, which ranges from about 2009 to the present, the most notable paradigm shift has been the movement from 'web apps' to true native apps. The old web app paradigm involved writing a web app in HTML, CSS, and JavaScript, that would be ported by some method, such as the now-obsolete PhoneGap,

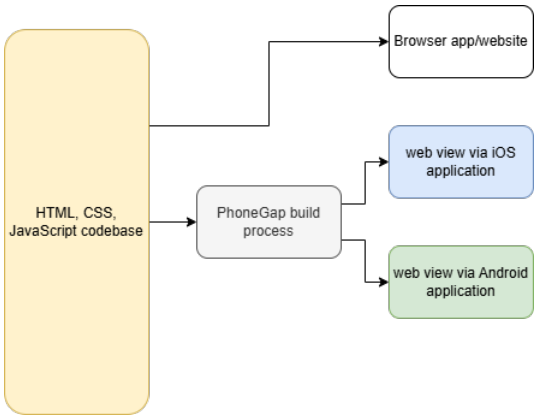

Figure 2: PhoneGap process

to a mobile application that would be downloaded via a native app store[6, 7]. These apps can be accessed from different platforms, and have the same look and feel on all devices, but lack the look and feel of a native app. The new cross-platform approach paradigm uses a framework such as React Native or Flutter to create a codebase that is either interpreted or cross-compiled to create native components on different platforms [8]. These apps are closer to the look and feel of truly native apps, but only require the development of one codebase. The downside of this approach is that the developer has to learn a new language and/or framework for this specific task. This evolution follows the adoption of mobile phones as stand-alone devices. The motivation for cross-platform mobile development has changed from creating convenient ports of web apps to developing mobile apps as their own product. To better understand this shift, one may consider how many applications on their mobile phone began as websites that would be (and in many cases still are) accessed via a browser.

The aforementioned PhoneGap, developed in 2008 by Nitobi Software, was the first major step in cross-platform development, and pioneered the 'web app' approach described above. Nitobi Software was acquired by Adobe in 2011, and PhoneGap became part of Apache Cordova [9, 10].

The first widely accepted approach for creating truly native cross-platform apps was React Native. Released in 2015, it extends the React JavaScript library and is written using the JSX extension for JavaScript [11]. The React Native framework is still actively supported and maintained by Meta, along with the open-source community. React Native is able to render native UI from common source code by interpreting JSX code to create truly native code that is used to render UI components [12].

This was closely followed by the release of Google's Flutter in 2017 [13]. Flutter apps are built using the Google's Dart programming language, and instead of compiling to native code, apps ship with their own rendering engine [14]. This approach has the benefit of being very performant, but the downside of not using truly native UI components. Flutter solves this problem by providing replicas of UI components for Android and iOS devices.

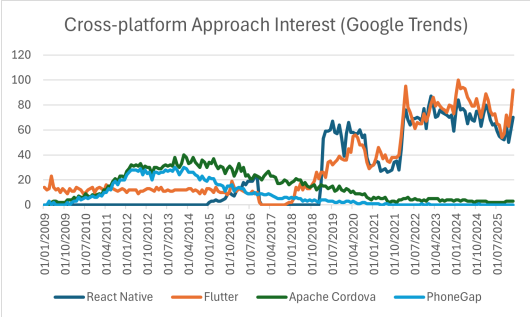

Figure 3: Search interest over time (data from [8])

2.2 Cross-Platform Approach Comparison

To deal with the problem of needing to develop applications for multiple different platforms, Heitkötter, Hanschke, and Majchrzak [15] define three kinds of approaches: Web-based development, native development, and cross-platform approaches. A brief summary of each follows.

- *Web development*: Applications are developed as browser apps, which are then accessed via the device's browser application
- *Native development*: Apps are created using the IDE associated with the target platform, e.g. Android Studio for Android, and XCode for iOS
- *Cross-platform approaches*: A third-party framework is used to create a single codebase that is compiled or interpreted to native code

For this paper, we will be focusing on cross-platform approaches, with an emphasis on the process of writing one codebase that can be used to develop for different platforms.

It is important to define the criteria that will be used to evaluate cross-platform approaches, so that qualitative and quantitative observations and comparisons can be made. Jošt and Taneski [16] utilise data from developer surveys, community engagement and usage, and job market research, along with data from Google Trends to provide a numerical comparison of different approaches. This view of market and developer trends is useful as it can help developers choose a framework based on its current support and community, or its value in the job market. Heitkötter, Hanschke, and Majchrzak based their evaluation on two sets of criteria one based on infrastructure, and one on development experience [15]. The infrastructure criteria involve the features offered by the approach, the community and support available, and the potential quality of the apps that can be built with it. The development criteria focuses on the development environment, learning process, and development features such as graphical UI editors and simulators. These criteria and approaches are used in Section 4 to develop a set of criteria to evaluate React Native and Flutter, and the audio that their apps produce.

2.3 Audio Applications

While there is a wealth of research done on the benefits, limitations, and options regarding cross-platform mobile development, there is a lack of information regarding audio development in this field. While audio processing is mentioned in some papers as an example of a complex real-time process that could require native code to reach performance requirements, there is no focus on the current tools and techniques for cross-platform audio development [17]. This is certainly an area in need of improvement, as mobile devices have become one of the most popular mode of listening to music, podcasts, audiobooks etc., along with becoming tools capable of complex audio recording and editing tasks.

When developing audio applications for iOS, the immediately available solution is AVFAudio [18]. This is Apple's built in audio library for playing, recording, and processing audio. In a simple application, such as a music player app, a developer may only need to declare an `AVAudioPlayer` object and pass it an audio file to begin playback. These high level objects offer a comfortable level of abstraction from the underlying audio graph, while still providing adequate control for simple applications. If the application calls for more advanced control, such as live processing or synthesis, AVFAudio provides access to the audio graph via the `AVAudioEngine`. The audio graph is a sequence of connected 'nodes' that take audio input, process it, and render it to an output. The developer can create nodes in this graph that synthesise and process audio. Real-time processing and effects need to be done as efficiently as possible, so they are written as C++ classes, with an Objective-C adapter class to allow the app's Swift code to communicate with them [19]. High level third-party libraries are also available for developers. The free and open-source AudioKit project provides the same recording, playback, and processing capabilities as AVFAudio, but with many more effects and functionalities built in [20]. AudioKit works by wrapping AVFAudio objects in a layer of abstraction, making it easier to build a complex audio graph, providing many useful effect and processing nodes so that the user does not need to create them from scratch. AudioKit also offers a graphical extension for that allows the user to create their processing chain with minimal code [21]. AudioKit objects do provide access to the underlying AVAudioNodes that they are build with.

Android audio development is similar to that of iOS, in that a build in library (android.media) is immediately available to the developer [22]. The user may only need to declare `MediaPlayer` and `MediaRecorder` objects for playback and recording, respectively. The `android.media` library does provide some basic audio effects, such as equalization and reverb. In the documentation for these effects it is recommended that developers use the premade effect classes rather than implementing their own using the `AudioEffect` class. Since Android apps are written in Java or Kotlin, and the underlying code that accesses hardware is written in C++, a JNI is necessary to access the device's hardware abstraction layer (HAL), as seen in fig. 5[23]. These layers of abstraction create latency and make the default audio api unsuitable for real-time processing [24].

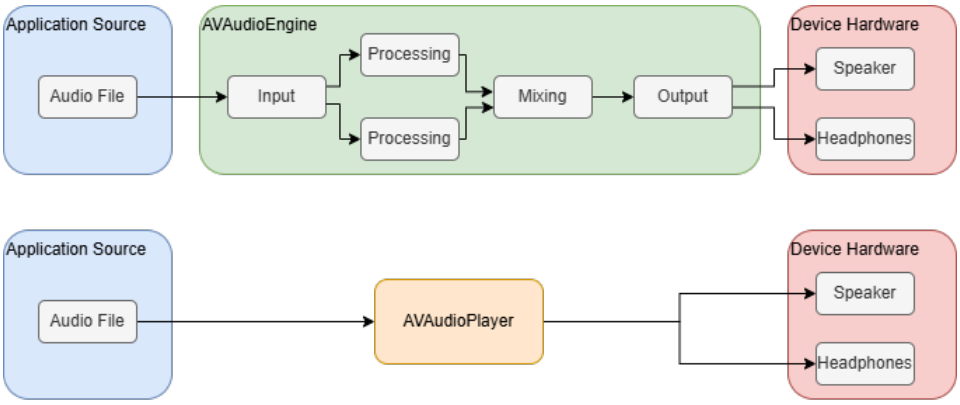

Figure 4: High level (top) and low level (bottom) audio control with AVFAudio

When using a cross platform approach, it is obviously much more difficult to have low-level control over audio. The differences in the hardware and the way audio is processed on different platforms is great enough that a high level of abstraction is required to create code that will work on multiple platforms. Luckily, there are many solutions available for platforms like React Native and Flutter, due to their large and active communities. From a review of some popular libraries, it seems that most offer a similar functionality to the native solutions, with recording, playback, and basic processing available at a high level [25, 26]. Research regarding the extend of low-level control and real-time processing that is possible will be conducted.

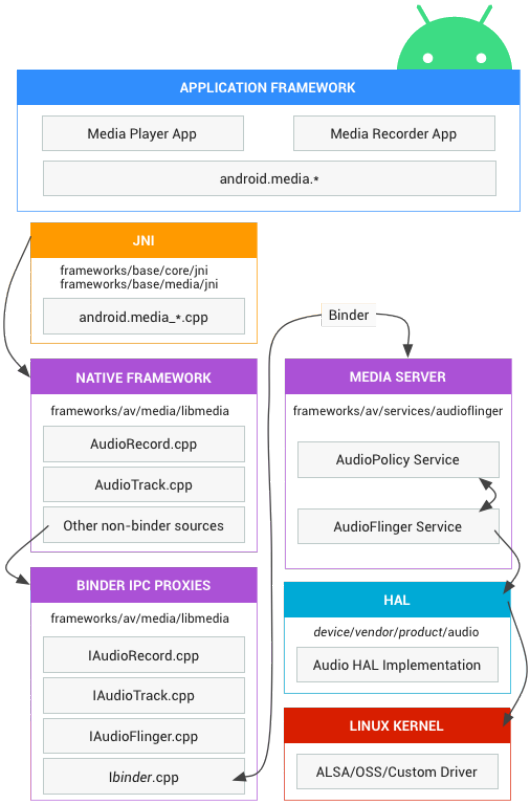

Figure 5: Android audio architecture (via [23])

3 Aims and Objectives

3.1 Aims

3.2 Objectives

4 Design

The design of the sample apps' audio processing will be implemened using the best practices for audio processing design [CITE].

Specifically, a circular buffer will be used to read data from the microphone, process it, and write it to the speaker. Audio will be stored using 16-bit PCM encoding. Audio will be stored using 32-bit floating point encoding. Audio will be processed in 10 millisecond chunks, with a sampling rate of 48 kilohertz.

4.1 Evaluation

Two sets of criteria have been developed to evaluate and compare React Native and Flutter, and the audio that they can produce. The framework criteria involves the infrastructure, development experience, and capabilities of each framework as seen in table 1. For audio, objective criteria will be used to measure latency and STOI scores, along with subjective feedback from test subjects as seen in table 2. The frameworks will also be evaluated using some objective statistics, specifically data from the 2025 StackOverflow Developer Survey [27]. These statistics will be taken into account as part of the 'Community and Support Criteria.

4.1.1 Framework Criteria

Table 1: Criteria for framework evaluation

Criteria	Description
Community and Support	Rating of the framework's 'popularity' based on the amount of available third-party support, documentation, and libraries/packages available
Features	Features that the framework can use in apps, e.g. native UI components, device-specific hardware access, etc.
Learning Curve	The time and effort required for a new user to start developing apps with the framework, and the amount of framework-specific knowledge that is required to use the framework
Environment Features	The development tools offereed by the framework, such as graphical UI editors, live editing, device simulators, and IDE extensions

The data taken from the 2025 StackOverflow Developer Survey gives some insight into the prevalence of different frameworks and the programming languages that they use, and in

turn the amount of support a developer can expect to find on StackOverflow. The question tag system on StackOverflow is also used to determine how many questions are being asked regarding each framework and its associated language.

4.1.2 Audio Criteria

Table 2: Criteria for audio evaluation

Criteria	Description
Latency	When processing live audio, the delay between input and output
Speech Intelligiblity	When processing speech, the STOI [CITE] of output audio compared to input audio
Subjective User Tests	Tests will be conducted with potential users to evaluate apps' audio quality

Audio quality will be objectively evaluated by measuring live processing latency and intelligibility. Latency can be simply measured by sending an impulse as an input, and measuring the time it takes for that impulse to be processed and played back. This process will be repeated multiple times and the results will be averaged. Non-live audio latency will be measured by measuring the time taken to process an audio file, taking into account the size of the audio file and any other settings regarding processing, e.g. sampling rate, and bit depth. Both of these measurements can be performed in the app's code, and could be used as a feature to provide information to the user.

5 Implementation

6 Testing

7 Evaluation and Further Work

7.1 Evaluation

7.2 Further Work

8 Recommendation

A References

References

[1] Statista. Statista number of smartphone users worldwide from 2014 to 2029. https://www.statista.com/forecasts/1143723/smartphone-users-in-the-world/. Accessed 2026.

[2] Ankush Chandrashekar, Pranav Vigneshwar Kumar, and B. R. Chandavarkar. Comparative analysis of modern mobile operating systems. In *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–7, 2021.

[3] StatCounter Global Stats. Operating system market share worldwide. https://www.statista.com/forecasts/1143723/smartphone-users-in-the-world/. Accessed 2026.

[4] Apple. Human interface guidlines - design - apple developer. https://developer.apple.com/human-interface-guidelines. Accessed 2026.

[5] Android Developers. Android developers. https://developer.apple.com/human-interface-guidelines. Accessed 2026.

[6] Lundrigan L Allen S., Graupera V. *Pro Smartphone Cross-Platform Development*, chapter 8. Apress, 2010.

[7] Ravindra Gangundi. Smartphone application development using cross platform frameworks. In *Proceedings of the National Conference on Information and Communication Technology, NMIMS University*, 2010.

[8] Google Trends. React native, flutter, phonegap, apache cordova interest over time. https://trends.google.com/explore?q=%2Fg%2F11h03gfxy9%2C%2Fg%2F11f03_rzbg%2C%2Fm%2F06znsr5%2CPhoneGap&date=2009-01-01%202026-03-02&geo=GB. Accessed 2026.

[9] Apache Cordova. Apache cordova. https://cordova.apache.org. Accessed 2026.

[10] ars Technica. "phonegap to become an apache project as adobe acquires nitobi". https://arstechnica.com/information-technology/2011/10/phonegap-to-become-an-apache-project-as-adobe-acquires-nitobi/. Accessed 2026.

[11] React. "react". https://react.dev. Accessed 2026.

[12] William Danielsson. *"React Native Application Development"*. PhD thesis, Linköping University, 2016.

[13] Ron Amadeo. "google starts a push for cross platform app development with flutter sdk". https://arstechnica.com/gadgets/2018/02/google-starts-a-push-for-cross-platform-app-development-with-flutter-sdk/. Accessed 2026.

[14] Dart. "introduction to dart". https://arstechnica.com/gadgets/2018/02/google-starts-a-push-for-cross-platform-app-development-with-flutter-sdk/. Accessed 2026.

[15] Majchrzak T.A. Heitkötter H., Hanschke S. "evaluating cross-platform development approaches for mobile applications". In Krempels KH Cordeiro J., editor, *Web Information Systems and Technologies*, volume 140. WEBIST, 2012.

[16] Taneski V. Jošt G. "state-of-the-art cross-platform mobile application development frameworks: A comparative study of market and developer trends". In *Informatics*, volume 12, 2025.

[17] Rohan Patil. "cross-platform mobile development: Strategic balancing of android and ios application design for optimal user experience and performance". In *Journal of Android iOS Development and Testing*, volume 10, 2025.

[18] AVFAudio. "avfaudio. https://developer.apple.com/documentation/avfaudio. Accessed 2026.

[19] Apple Developer Documentation. "creating custom audio effects". https://developer.apple.com/documentation/avfaudio/creating-custom-audio-effects. Accessed 2026.

[20] AudioKit. Audiokit. https://github.com/AudioKit/AudioKit. Accessed 2026.

[21] AudioKit. Flow. https://github.com/AudioKit/Flow/. Accessed 2026.

[22] Android Developers. "android.media". https://developer.android.com/reference/android/media/package-summary.html. Accessed 2026.

[23] Android Open Source Project. "audio". https://source.android.com/docs/core/audio. Accessed 2026.

[24] Luca Abeni Alessio Balsini, Tommaso Cucinotta. "energy-efficient low-latency audio on android". In *Journal of Systems and Software*, volume 152, pages 182–195, 2019.

[25] Software Mansion. "react native audio api". https://docs.swmansion.com/react-native-audio-api/. Accessed 2026.

[26] pub.dev. "audioplayers 6.6.0". https://pub.dev/packages/audioplayers. Accessed 2026.

[27] Stack Overflow. Technology | 2025 stackoverflow developer survey. https://survey.stackoverflow.co/2025/technology/#most-popular-technologies. Accessed 2026.

B Figures